

Human Factors and Usable Security

Dr. Ramokapane

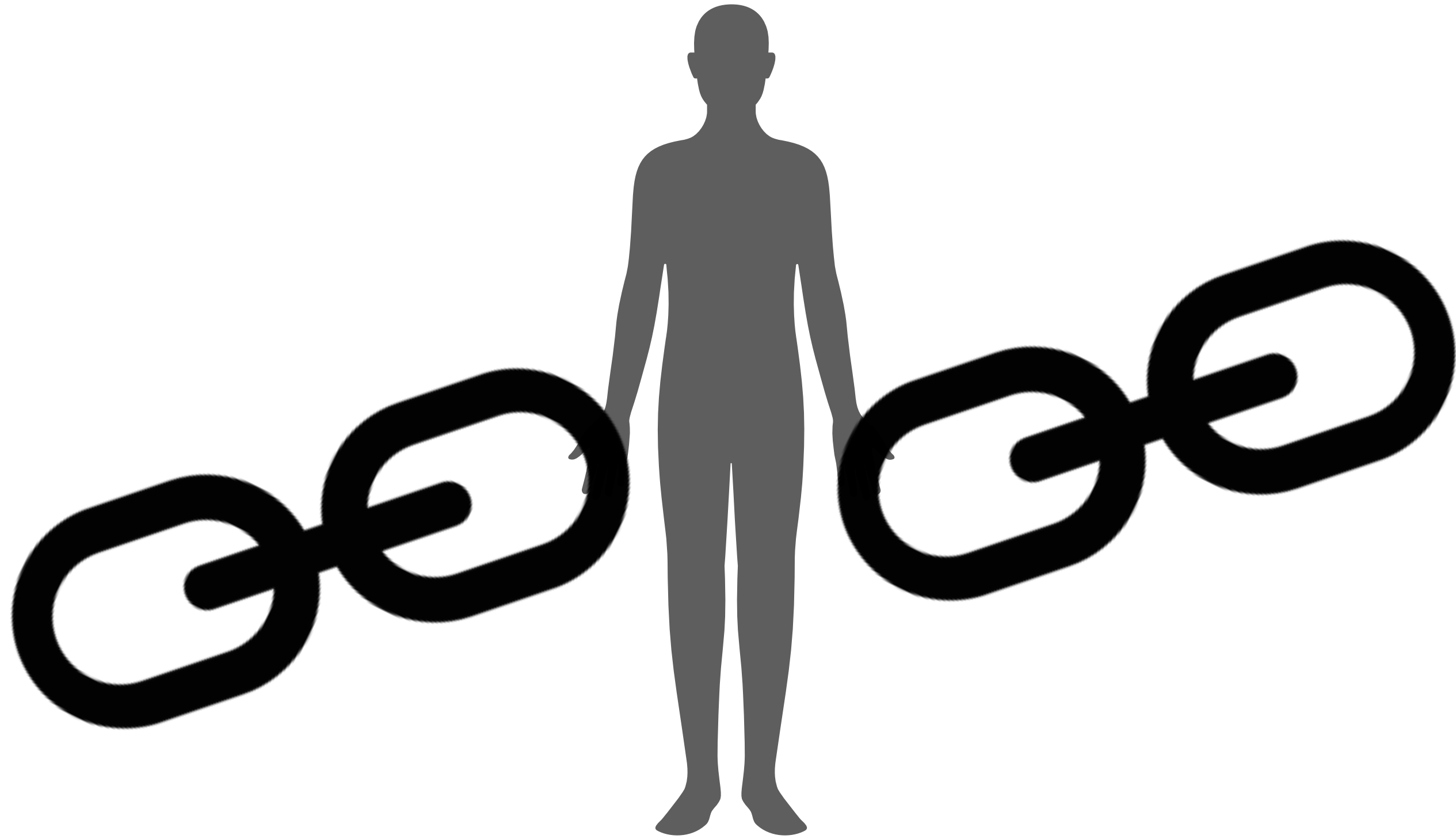


Bristol Cyber Security Group

Less than 0.1% of email is end-to-end encrypted!

A. Whitten and J. D. Tygar, “Why johnny can’t encrypt: A usability evaluation of pgp 5.0.” in USENIX Security Symposium, vol. 348, 1999.





Users are not the Enemy!

A. Adams and M. A. Sasse, “Users are not the enemy,” Communications of the ACM, vol. 42, no. 12, pp. 40–46, 1999.

Fit the human to the task



Fit the task to the human

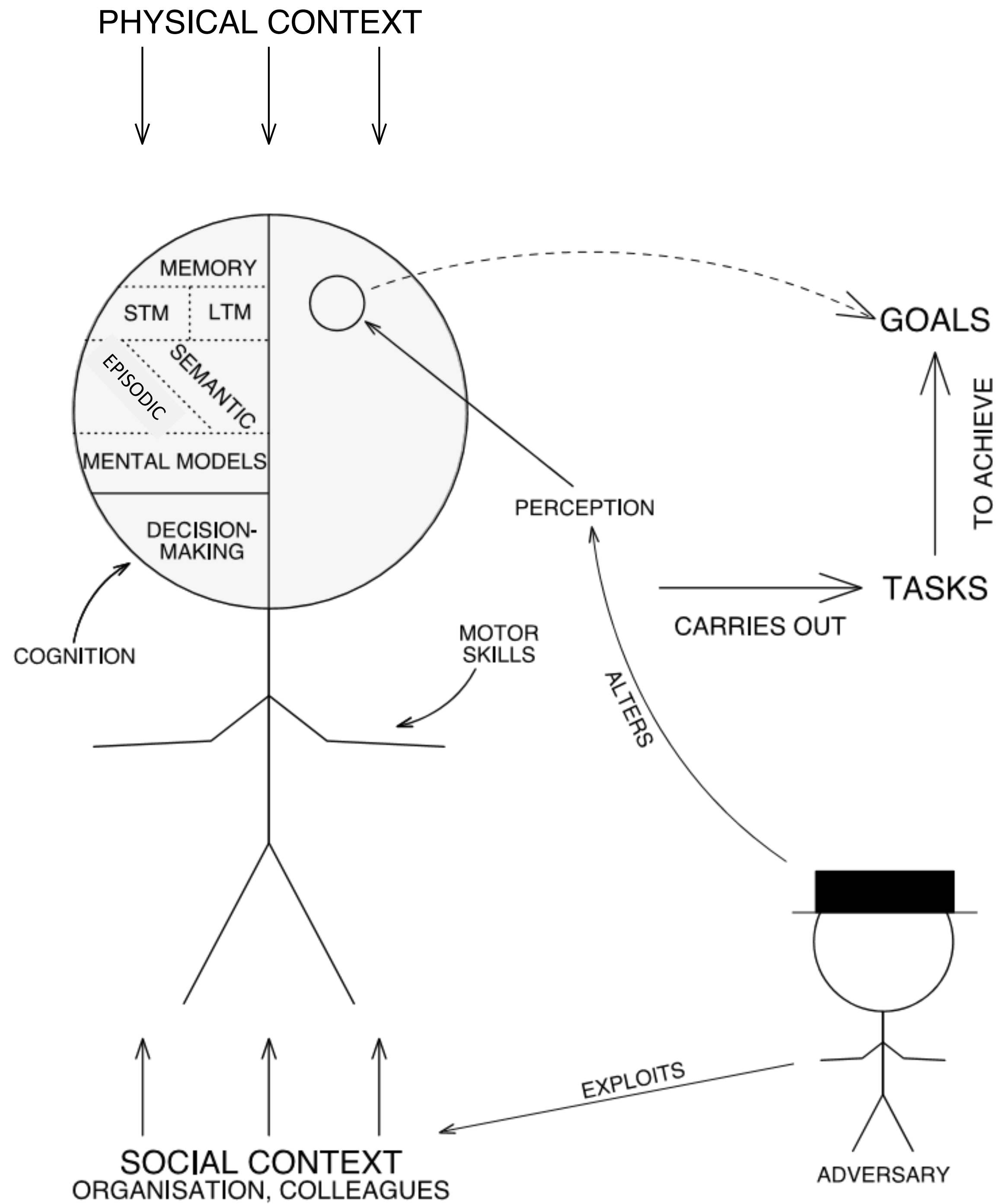
Security has to be usable

Effectiveness: Can users achieve their goals?

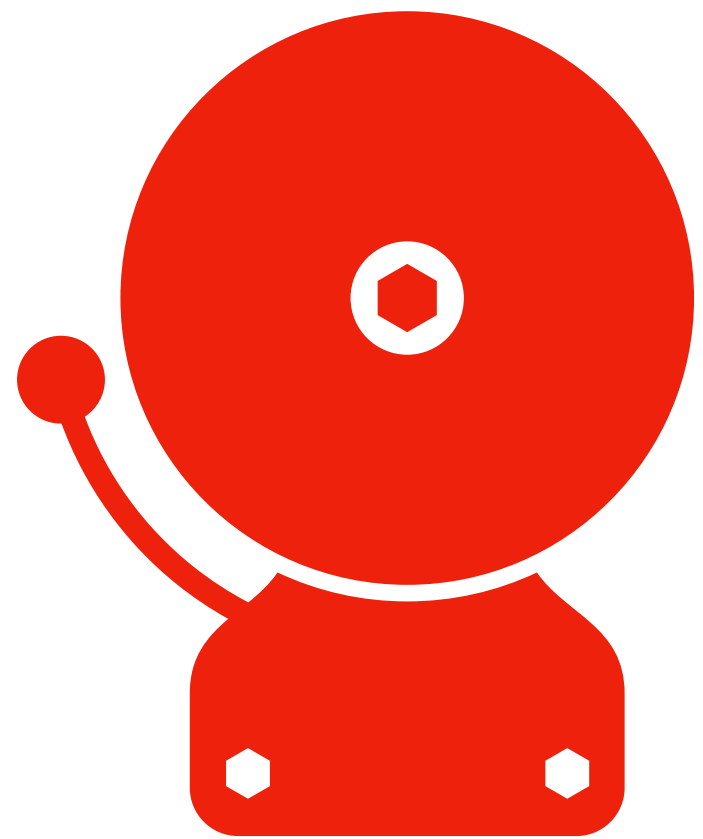
Efficiency: What resources are expended to do so?

Satisfaction: What is the level of comfort and acceptability for users?

HUMAN CAPABILITIES & LIMITATIONS



Human Capabilities and Limitations



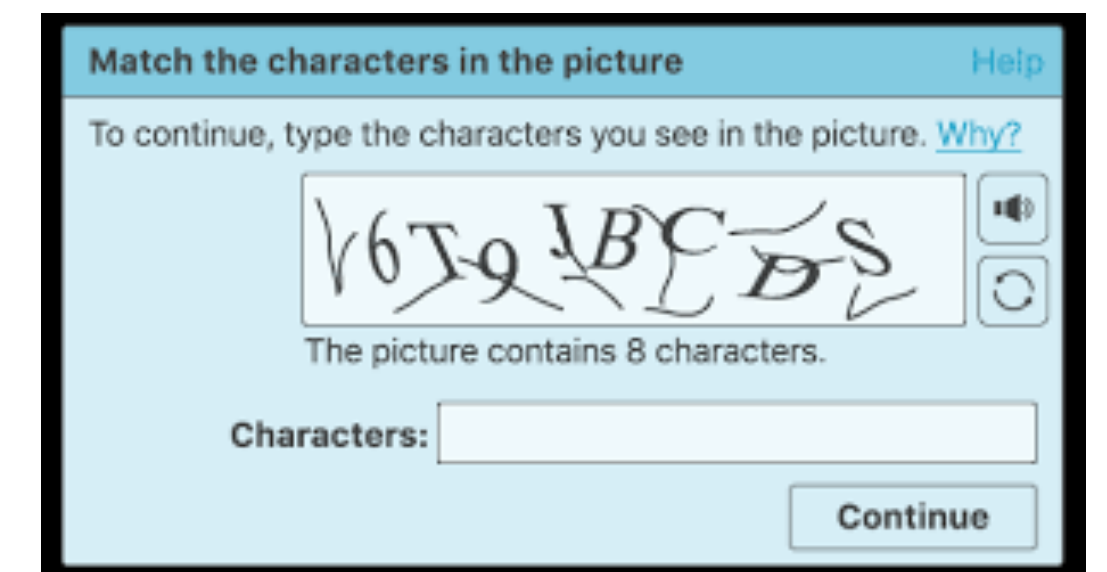
Alarm Fatigue



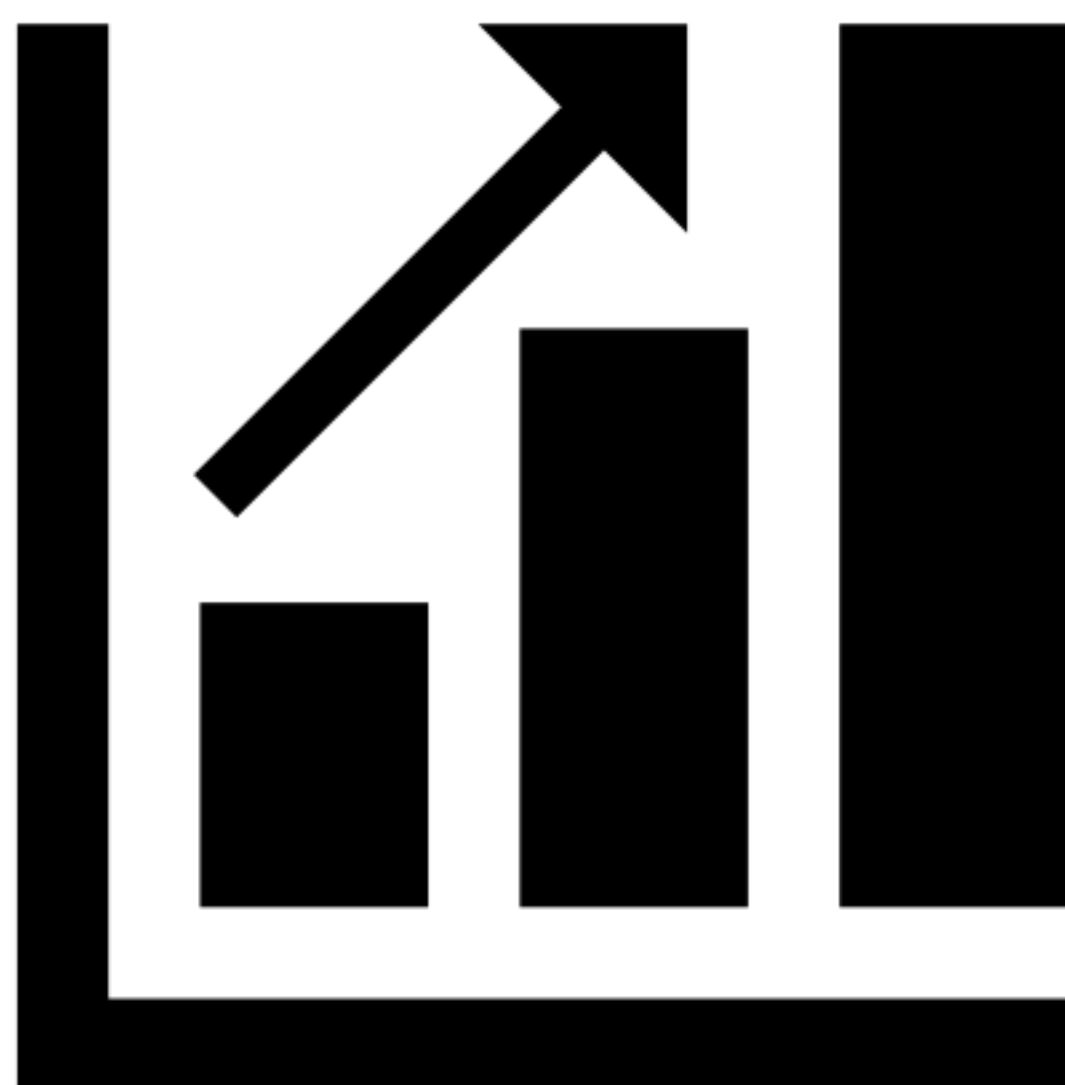
STM vs. LTM



Strength Meters



CAPTCHA



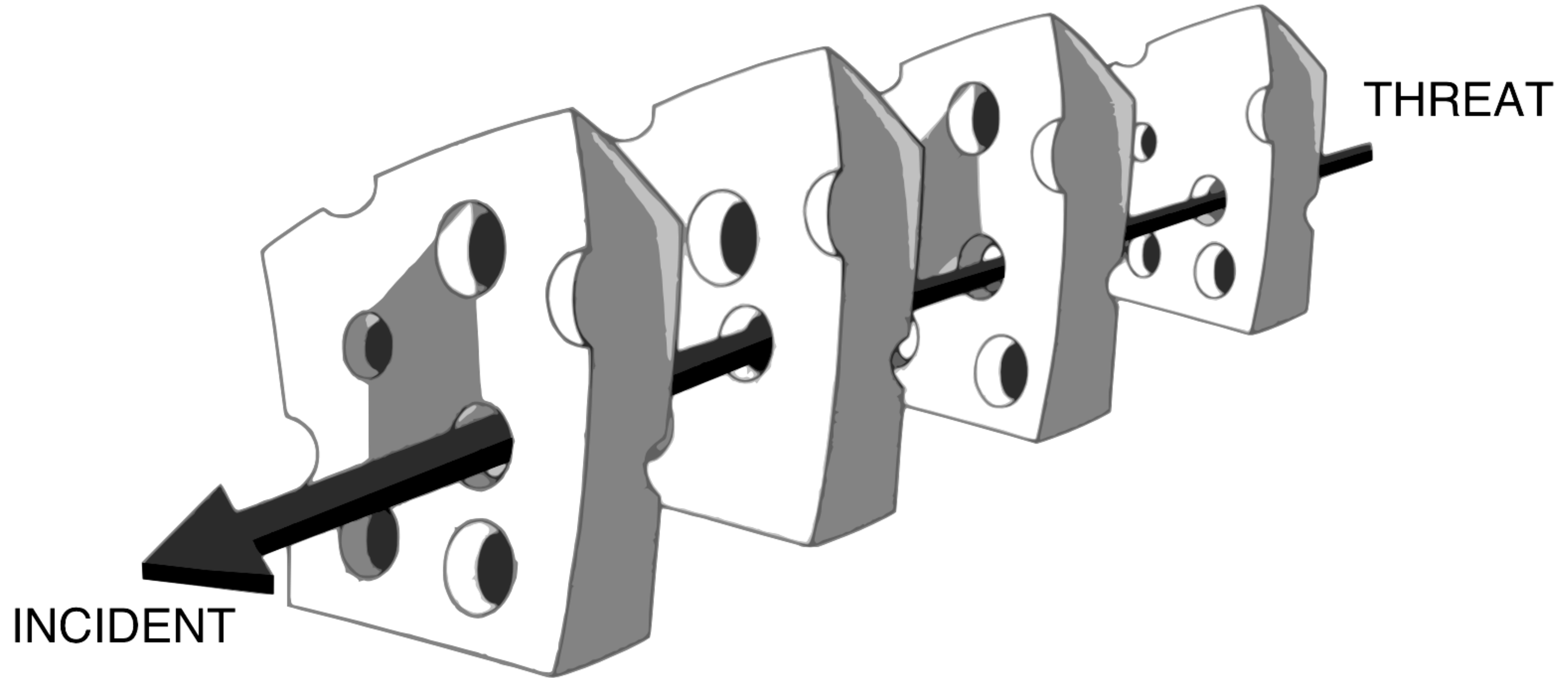
vs.





Compliance Fatigue

Latent Design Conditions



“Never give an order that can’t be obeyed”

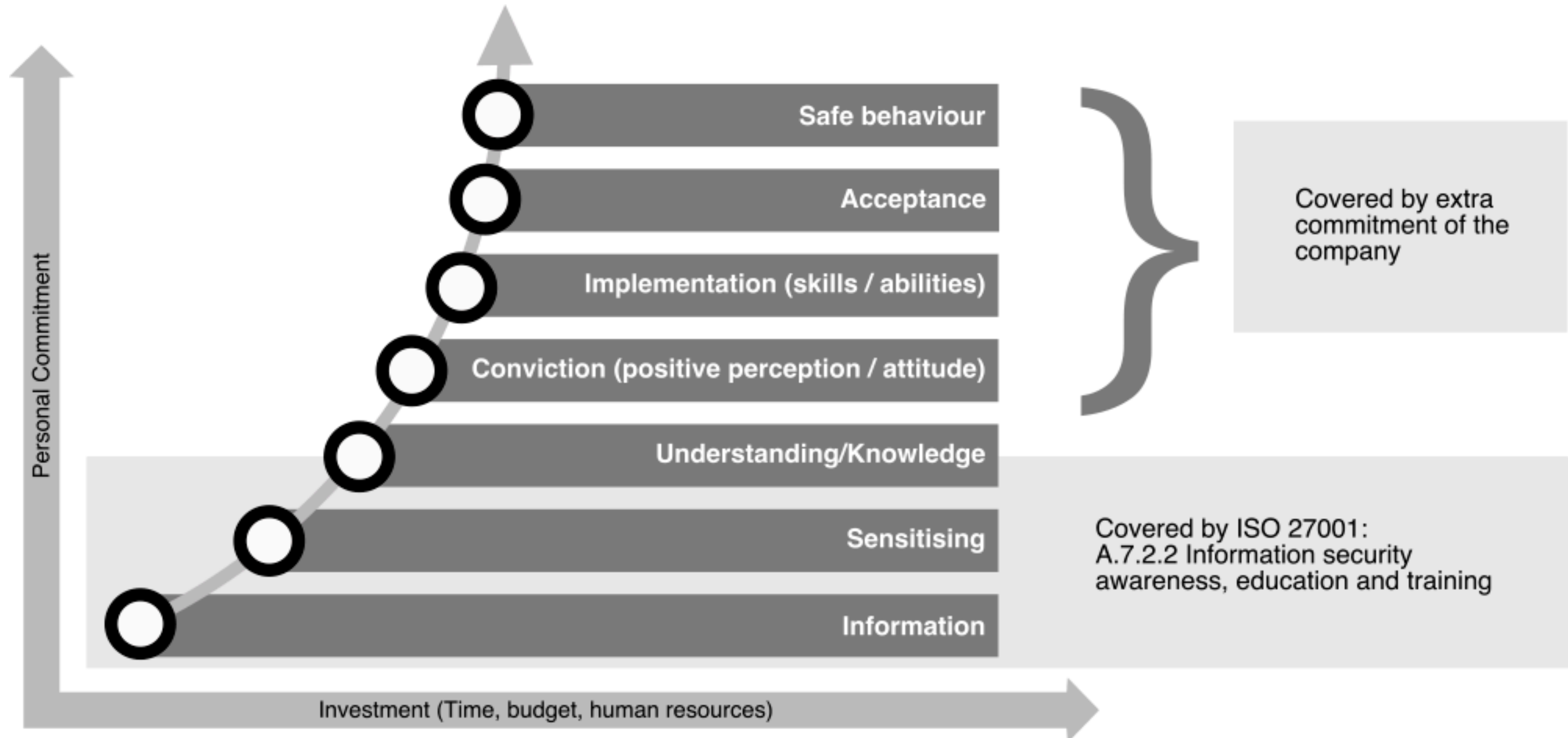
General MacArthur

Never issue a security policy that cannot be followed

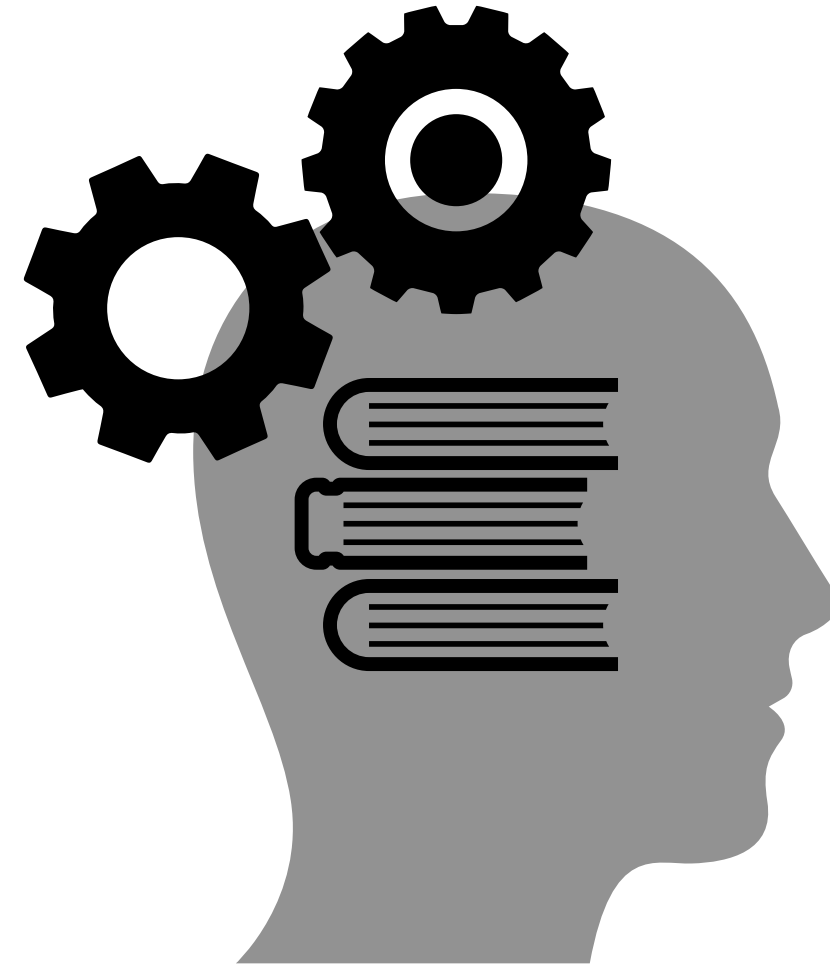


Shadow Security

Awareness and education



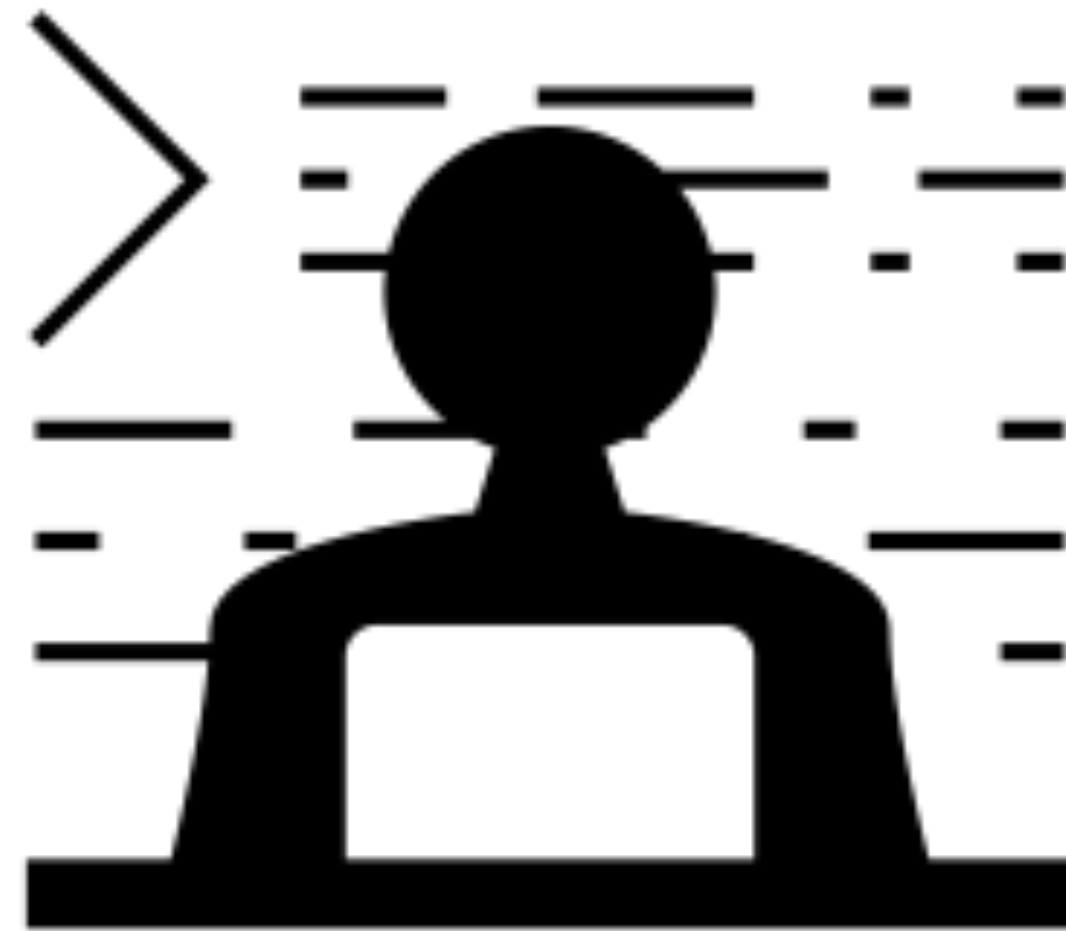




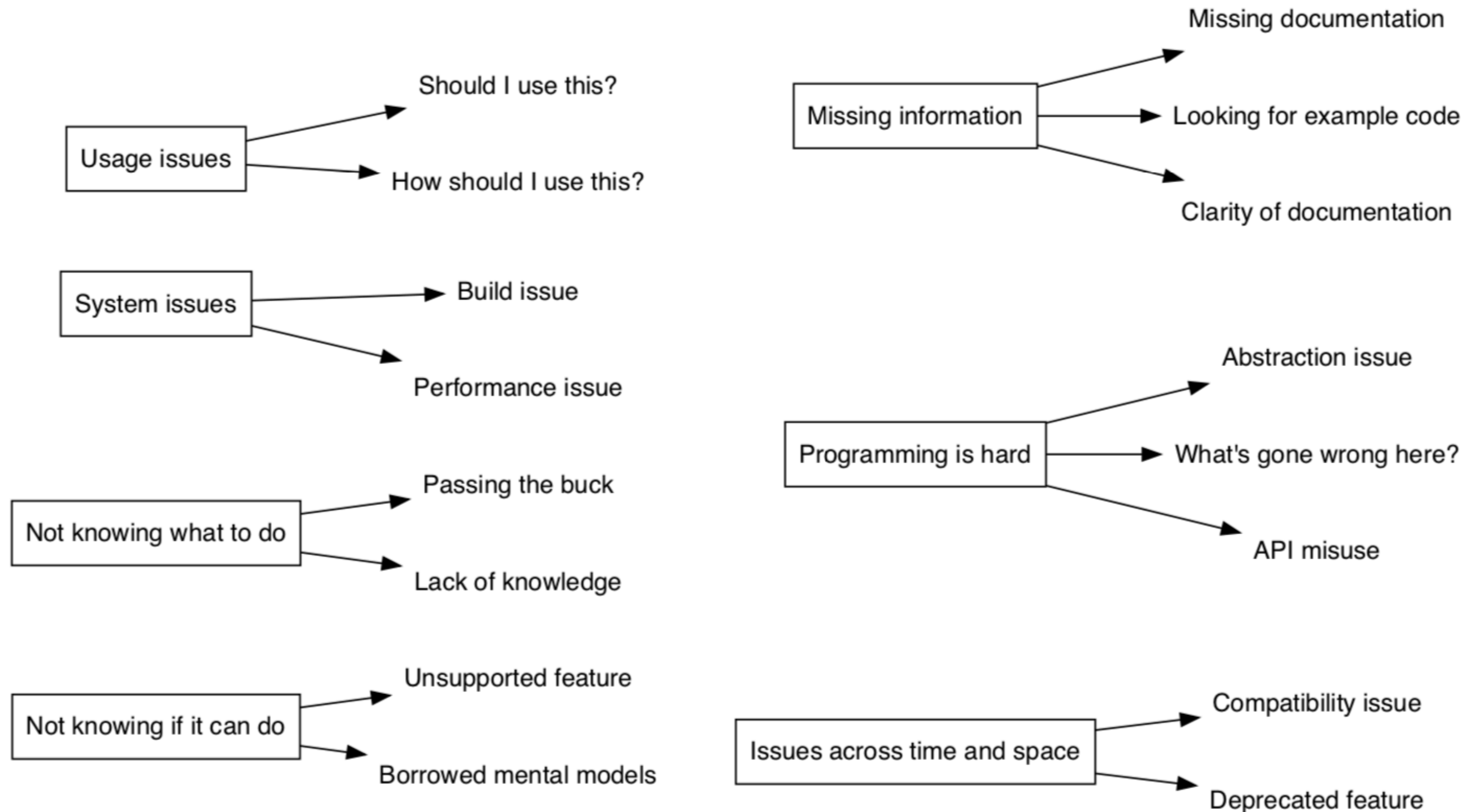
**Mental models
of cyber security
and risks**



FUD



What about developers?



N. Patnaik, J. Hallett, and A. Rashid, “Usability smells: An analysis of developers’ struggle with crypto libraries,” in Fifteenth Symposium on Usable Privacy and Security (SOUPS), Santa Clara, USA. USENIX Association, 2019.

Developers are Not the Enemy!

The Need for Usable Security APIs

Matthew Green and Matthew Smith
IEEE Security & Privacy 14(5), Sept.-Oct. 2016.

Principles for Usability of Crypto APIs

Abstract: Integrate cryptographic functionality into standard APIs so regular developers do not have to interact with cryptographic APIs in the first place.

Powerful: Sufficiently powerful to satisfy both security and non-security requirements.

Comprehensible: Easy to learn, even without cryptographic expertise.

Ergonomic: Don't break the developer's paradigm.

Intuitive: Easy to use, even without documentation.

Principles for Usability of Crypto APIs

Failing: Hard to misuse. Incorrect use should lead to visible errors.

Safe: Defaults should be safe and never ambiguous.

Testable: Testing mode. If developers need to run tests they can reduce the security for convenience.

Readable: Easy to read and maintain code that uses it/Updatability.

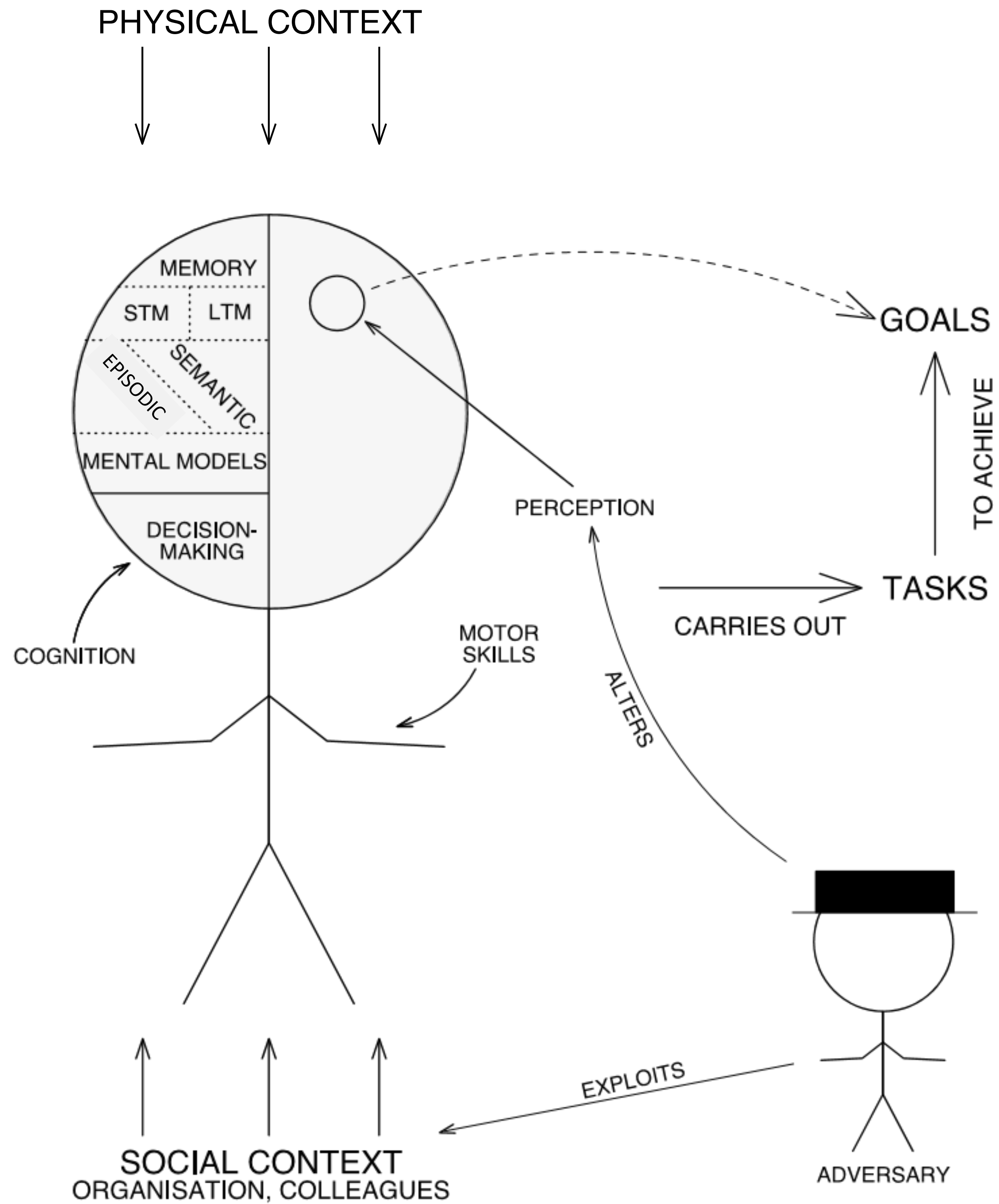
Explained: Assist with/handle end-user interaction, and provide error messages where possible.

Usability *smells*

Whiff	Issue	Abstract	Powerful	Comprehensible	Ergonomic	Intuitive	Failing	Safe	Testable	Readable	Explained
Need a super-sleuth	Missing Documentation				●						
	Example code				●		●				
	Clarity of documentation				●		●				
Confusion reigns	Should I use this?			●		●					
	How should I use this?	●	●			●					
	Abstraction issues	●				●					
	Borrowed mental models				●						
Needs a post-mortem	What's gone wrong here?				●	●				●	●
	Unsupported feature					●					
	API misuse						●	●			
	Deprecated feature						●				
Doesn't play well with others	Build issues								●		
	Compatibility issues		●								
	Performance issues										

N. Patnaik, J. Hallett, and A. Rashid, “Usability smells: An analysis of developers’ struggle with crypto libraries,” in Fifteenth Symposium on Usable Privacy and Security (SOUPS), Santa Clara, USA. USENIX Association, 2019.

HUMAN CAPABILITIES & LIMITATIONS



Fit the human to the task



Fit the task to the human

Further Reading

Listen to this podcast: <https://www.cybok.org/events/human-factors-podcast>

Human Factors Knowledge Area in the Cyber Security Body of Knowledge

https://www.cybok.org/media/downloads/Human_Factors_v1.0.1.pdf